

n^2 steps to perform, as n grows very large, the difference between them will matter less and less. Both of them run in more or less n^2 steps.

Using Big O Notation

To say that an algorithm runs in more or less n^2 steps is a reasonable balance between the precision we want and the conciseness we want (and the randomness we have). The way we express this type of “more or less” relationship formally is by using *big O* notation (the O is short for *order*). We might say that a particular algorithm is “big O of n^2 ,” or $O(n^2)$, if, in the worst case, it runs in more or less n^2 steps for large n . The technical definition states that the function $f(x)$ is big- O of the function $g(x)$ if there’s some constant number M such that the absolute value of $f(x)$ is always less than M times $g(x)$ for all sufficiently large values of x .

In the case of insertion sort, when we look at our expression for the maximum number of steps required to perform the algorithm, we find that it’s a sum of two terms: one is a multiple of n^2 , and the other is a multiple of n . As we just discussed, the term that is a multiple of n will matter less and less as n grows, and the n^2 term will come to be the only one that we are concerned with. So the worst case of insertion sort is that it is a $O(n^2)$ (“big O of n^2 ”) algorithm.

The quest for algorithm efficiency consists of seeking algorithms whose runtimes are big O of smaller and smaller functions. If we could find a way to alter insertion sort so that it is $O(n^{1.5})$ instead of $O(n^2)$, that would be a major breakthrough that would make a huge difference in runtimes for large values of n . We can use big O notation to talk not only about time but also about space. Some algorithms can gain speed by storing big datasets in memory. They might be big O of a small function for runtime but big O of a larger function for memory requirements. Depending on the circumstances, it may be wise to gain speed by eating up memory, or to free up memory by sacrificing speed. In this chapter, we’ll focus on gaining speed and designing algorithms to have runtimes that are big O of the smallest possible functions, without regard to memory requirements.